

Fundamentals of Database

CBIO204

Biomedical

Name	ID
Hana Hisham Saad	241000853
Menatalla Ahmed Osman	211000170

Under the supervision of:

- Dr. Mohamed Mahmoud Elsayeh
- Eng. Habiba Mohamed Ibrahim

Spring 2026

1. Conceptual design: list of entities & attributes:

Conceptual Design

Entities & Attributes

Journal

- ISSN
- JournalName
- Impactfactor
- Quarter
- PublisherID
- Country
- StartDate
- Volume/PublicationRate
- OpenAccessStatus

Research

- ResearchID
- Title
- Abstract
- PublicationDate
- Citations
- BiomedicalFieldID
- JournalID

ResearchAuthor

- ResearchID
- AuthorID

Publisher

- PublisherID
- PublisherName

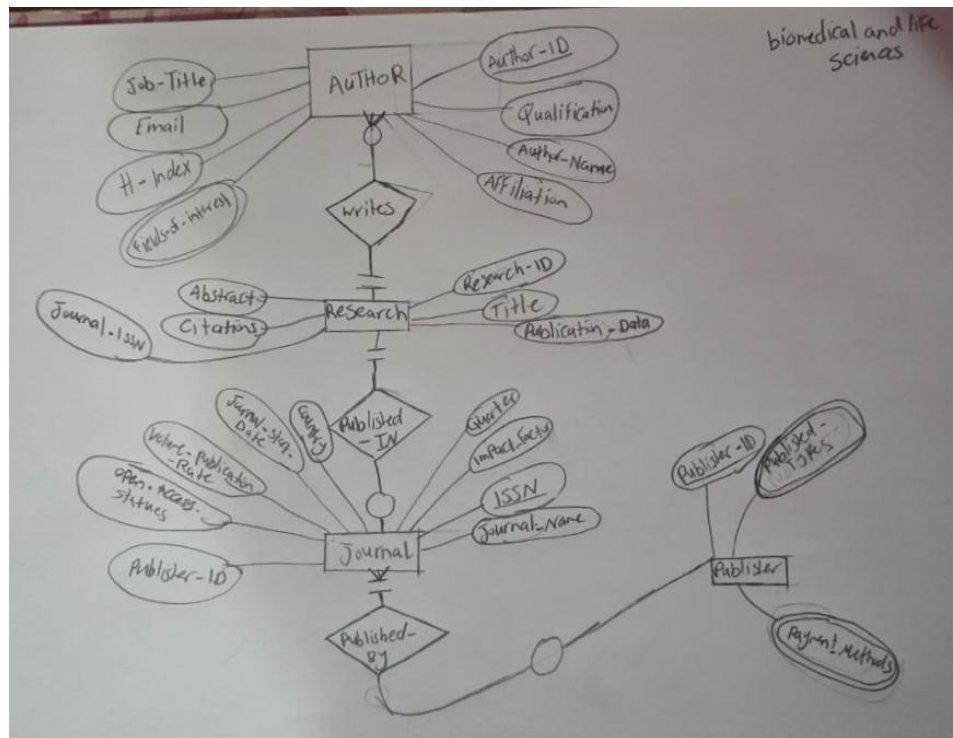
BiomedicalField

- FieldID

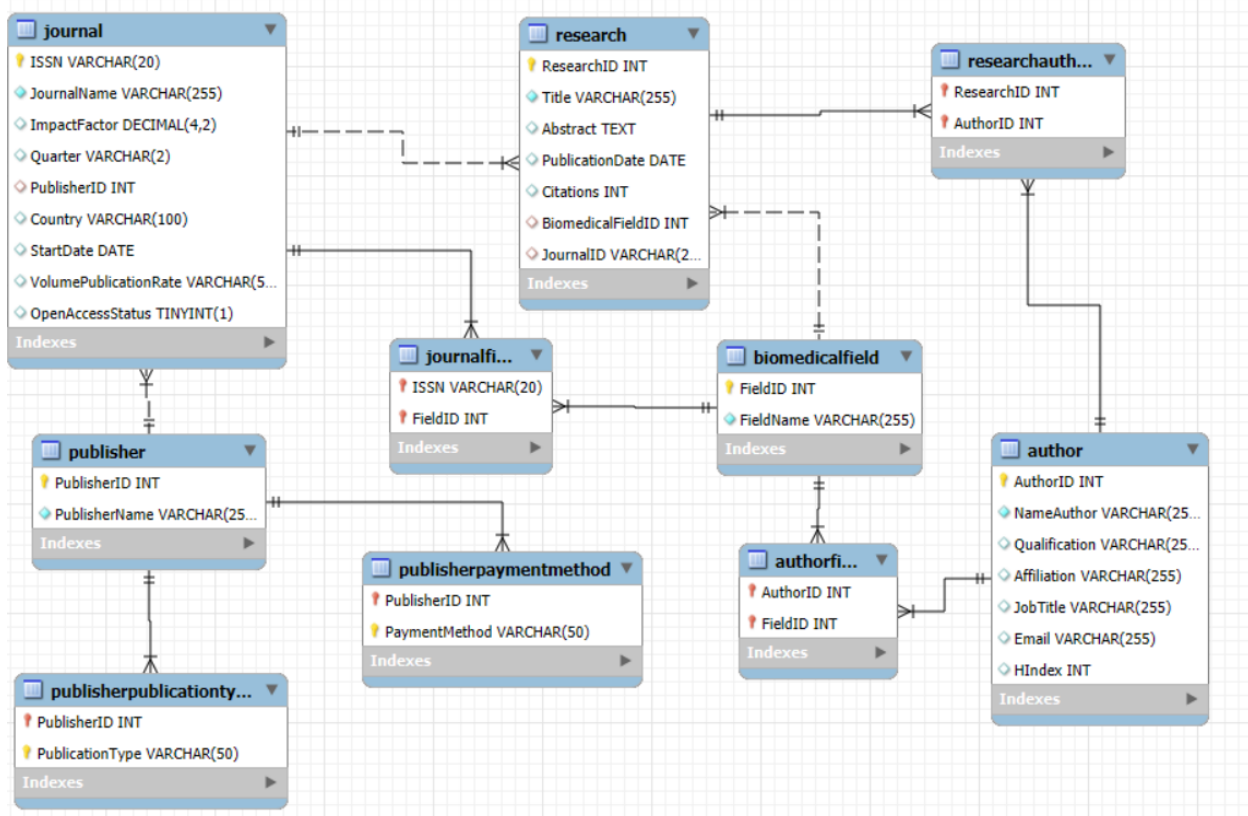
Author

- AuthorID
- NameAuthor
- Qualification
- Affiliation
- JobTitle

2. ER diagram: show cardinality ratios & participation constraints:



3. Relational schema: tables, attributes & constraints (PKs, FKs):



4. DDL & DML:

Database Creation

- sql code:

```
CREATE DATABASE Biomedical_1;
```

- Meaning:

Create a new database called Biomedical_1

Switch to using Biomedical_1:

- sql code:

```
USE Biomedical_1;
```

- Meaning:

Switch and use this database for the time being

Table Creation and Execution Flow:

-BiomedicalField table:

- sql code:

```
CREATE TABLE BiomedicalField (  
    FieldID INT PRIMARY KEY,  
    FieldName VARCHAR(255) NOT NULL  
);
```

- Meaning:

Creates a table to store biomedical research fields where 'FieldID' is the Primary Key and 'FieldName' is text, required which cannot be left alone hence the usage of 'NOT NULL'.

-Author table:

- sql code:

```
CREATE TABLE Author (  
    AuthorID INT PRIMARY KEY,  
    NameAuthor VARCHAR(255) NOT NULL,  
    Qualification VARCHAR(255),  
    Affiliation VARCHAR(255),  
    JobTitle VARCHAR(255),  
    Email VARCHAR(255) UNIQUE,  
    HIndex INT  
);
```

- Meaning:

It stores author details. 'AuthorID' is the primary key for each author. 'Email' must be unique (no duplicates). 'HIndex' is a numeric measure of research impact.

-Publisher table:

- sql code:

```
CREATE TABLE Publisher (  
    PublisherID INT PRIMARY KEY,  
    PublisherName VARCHAR(255) NOT NULL  
);
```

- Meaning:

It holds publisher information. 'PublisherID' is primary key for each publisher.

-Journal table:

- sql code:

```
CREATE TABLE Journal (  

```

```

ISSN VARCHAR(20) PRIMARY KEY,
JournalName VARCHAR(255) NOT NULL,
ImpactFactor DECIMAL(4,2),
Quarter VARCHAR(2),
PublisherID INT,
Country VARCHAR(100),
StartDate DATE,
VolumePublicationRate VARCHAR(50),
OpenAccessStatus BOOLEAN,
FOREIGN KEY (PublisherID) REFERENCES Publisher (PublisherID)
);

```

- Meaning:

It stores journal details. 'ISSN' is the primary key. 'PublisherID' links each journal to its publisher which is 'FOREIGN KEY' and ensures referential integrity. 'ImpactFactor' is a decimal value. 'OpenAccessStatus' is a Boolean (Yes/No).

-Research table:

- sql code:

```

CREATE TABLE Research (
    ResearchID INT PRIMARY KEY,
    Title VARCHAR(255) NOT NULL,
    Abstract TEXT,
    PublicationDate DATE,
    Citations INT,
    BiomedicalFieldID INT,
    JournalID VARCHAR(20),

```

FOREIGN KEY (BiomedicalFieldID) REFERENCES BiomedicalField(FieldID),

FOREIGN KEY (JournalID) REFERENCES Journal(ISSN)

);

- Meaning:

It stores research papers. It's linked to 'BiomedicalField' and 'Journal' via foreign keys. 'Citations' counts how many times the paper has been cited.

Junction Tables (Many-to-Many Relationships):

-ResearchAuthor table:

- sql code:

```
CREATE TABLE ResearchAuthor (
```

```
    ResearchID INT,
```

```
    AuthorID INT,
```

```
    PRIMARY KEY (ResearchID, AuthorID),
```

```
    FOREIGN KEY (ResearchID) REFERENCES Research(ResearchID),
```

```
    FOREIGN KEY (AuthorID) REFERENCES Author(AuthorID)
```

```
);
```

- Meaning:

It connects research papers to multiple authors. Composite Primary Key ensures no duplicate author-paper pair.

-AuthorField table:

- sql code:

```
CREATE TABLE AuthorField (
```

```
    AuthorID INT,
```

```
FieldID INT,  
PRIMARY KEY (AuthorID, FieldID),  
FOREIGN KEY (AuthorID) REFERENCES Author(AuthorID),  
FOREIGN KEY (FieldID) REFERENCES BiomedicalField(FieldID)  
);
```

- Meaning:

It links authors to multiple biomedical fields of interest.

-JournalField table:

- sql code:

```
CREATE TABLE JournalField (  
ISSN VARCHAR(20),  
FieldID INT,  
PRIMARY KEY (ISSN, FieldID),  
FOREIGN KEY (ISSN) REFERENCES Journal(ISSN),  
FOREIGN KEY (FieldID) REFERENCES BiomedicalField(FieldID)  
);
```

- Meaning:

It associates journals with multiple biomedical fields.

Publisher Extensions:

-PublisherPublicationType table:

- sql code:

```
CREATE TABLE PublisherPublicationType (  
PublisherID INT,
```



```
PublicationType VARCHAR(50), -- Journal, Book, eBook
PRIMARY KEY (PublisherID, PublicationType),
FOREIGN KEY (PublisherID) REFERENCES Publisher(PublisherID)
);
```

- Meaning:

It defines what types of publications a publisher produces.

-PublisherPaymentMethod table:

- sql code:

```
CREATE TABLE PublisherPaymentMethod (
    PublisherID INT,
    PaymentMethod VARCHAR(50),
    PRIMARY KEY (PublisherID, PaymentMethod),
    FOREIGN KEY (PublisherID) REFERENCES Publisher(PublisherID)
);
```

- Meaning:

It stores accepted payment methods for each publisher.

Insert:

-BiomedicalField table:

- sql code:

```
INSERT INTO BiomedicalField (FieldID, FieldName) VALUES
(1, 'Cell Biology'),
(2, 'Genetics'),
...
```

(10, 'Genomics');

- Meaning:

It adds 10 biomedical fields. Each row has a unique 'FieldID' and descriptive 'FieldName'. SQL executes this by inserting one row per '(FieldID, FieldName)' pair.

-Author table:

- sql code:

```
INSERT INTO Author (AuthorID, Name, Qualification, Affiliation, JobTitle, Email, HIndex)
VALUES
```

```
(1, 'Alice Johnson', 'PhD', 'Harvard University', 'Professor', 'alice@harvard.edu', 45),
```

...

```
(10, 'Jack Wilson', 'PhD', 'University of Chicago', 'Postdoc', 'jack@uchicago.edu', 12);
```

- Meaning:

It adds 10 authors with details like name, qualification, affiliation, and email. 'Email' must be unique and SQL enforces this constraint. 'HIndex' is a numeric measure of research impact.

-Publisher table:

- sql code:

```
INSERT INTO Publisher (PublisherID, PublisherName) VALUES
```

```
(1, 'Springer'),
```

...

```
(10, 'Royal Society of Chemistry');
```

- Meaning:

It adds 10 publishers. Each publisher has a unique 'PublisherID'.

-Journal table:

- sql code:

```
INSERT INTO Journal (ISSN, JournalName, ImpactFactor, Quarter, PublisherID, Country,
StartDate, VolumePublicationRate, OpenAccessStatus) VALUES
```

```
('J001', 'Nature Genetics', 45.67, 'Q1', 5, 'UK', '1990-01-01', 'Monthly', 1),
```

...

```
('J010', 'Pharmacology Review', 9.45, 'Q4', 4, 'USA', '1980-01-01', 'Quarterly', 0);
```

- Meaning:

It adds 10 journals. Each journal is linked to a publisher via 'PublisherID'. 'ImpactFactor' is a decimal value. 'OpenAccessStatus' is Boolean (1 = Yes, 0 = No).

-Research table:

- sql code:

```
INSERT INTO Research (ResearchID, Title, Abstract, PublicationDate, Citations,
BiomedicalFieldID, JournalID) VALUES
```

```
(1, 'Gene Editing Advances', 'Study on CRISPR applications.', '2020-05-01', 120, 2, 'J001'),
```

...

```
(10, 'Genomic Sequencing', 'Advances in sequencing tech.', '2023-06-18', 200, 10, 'J007');
```

- Meaning:

It adds 10 research papers. Each paper links to a **BiomedicalField** and a **Journal** via foreign keys. SQL checks that 'BiomedicalFieldID' and 'JournalID' exist before inserting.

Junction Tables (Many-to-Many Relationships):

-ResearchAuthor table:

- sql code:

```
INSERT INTO ResearchAuthor (ResearchID, AuthorID) VALUES
```

```
(1,1),(1,2),(2,3),(2,4),(3,5),(3,6),(4,7),(4,8),(5,9),(5,10);
```

- Meaning:

It connects research papers to authors. Example: Research 1 is authored by Author 1 and Author 2. SQL enforces that both IDs exist in their respective tables.

-AuthorField table:

- sql code:

```
INSERT INTO AuthorField (AuthorID, FieldID) VALUES
```

```
(1,2),(2,3),(3,4),(4,5),(5,1),(6,6),(7,7),(8,8),(9,9),(10,10);
```

- Meaning:

It links authors to their biomedical fields of interest. Example: Author 1 is interested in Genetics (Field 2).

-JournalField table:

- sql code:

```
INSERT INTO JournalField (ISSN, FieldID) VALUES
```

```
('J001',2),('J002',1),('J003',3),('J004',4),('J005',9),
```

```
('J006',8),('J007',10),('J008',7),('J009',6),('J010',5);
```

- Meaning:

It associates journals with biomedical fields. Example: Journal J001 covers Genetics (Field 2).

Publisher Extensions:

-PublisherPublicationType table:

- sql code:

```
INSERT INTO PublisherPublicationType (PublisherID, PublicationType) VALUES
```

```
(1,'Journal'),(2,'Journal'),(3,'Book'),(4,'Journal'),(5,'Journal'),
```

(6,'Book'),(7,'Book'),(8,'Journal'),(9,'Journal'),(10,'Journal');

- Meaning:

It defines what types of publications each publisher produces. Example: Publisher 3 produces Books.

-PublisherPaymentMethod table:

- sql code:

```
INSERT INTO PublisherPaymentMethod (PublisherID,  
PaymentMethod) VALUES
```

```
(1,'Cash'),(2,'Bank Transfer'),(3,'Online  
Payment'),(4,'Cash'),(5,'Online Payment'),
```

```
(6,'Bank Transfer'),(7,'Cash'),(8,'Online Payment'),(9,'Bank  
Transfer'),(10,'Cash');
```

- Meaning:

It stores accepted payment methods for each publisher.
Example: Publisher 2 accepts Bank Transfer.

DML queries:

- 1) Select authors with h-index greater than 25:

- sql code:

```
SELECT NameAuthor, HIndex
```

```
FROM Author
```

```
WHERE HIndex > 25;
```

- 2) Select journals that are open access:

- sql code:

```
SELECT JournalName
```

```
FROM Journal
```

```
WHERE OpenAccessStatus = 1;
```

```
209 • SELECT NameAuthor, HIndex  
210 FROM Author  
211 WHERE HIndex > 25;  
212
```

Result Grid		Filter Rows:
NameAuthor	HIndex	
Alice Johnson	45	
Bob Smith	32	
Carol White	28	
Eva Green	50	
Henry Adams	40	

```
213 • SELECT JournalName  
214 FROM Journal  
215 WHERE OpenAccessStatus = 1;
```

Result Grid		Filter Rows:
JournalName		
Nature Genetics		
Cell		
Development		
Genomics Journal		
Microbiology Today		

3) Update the impact factor of a journal:

2	Bob Smith	MD	University of California, Berkeley	Researcher	bob@stanford.edu	32
---	-----------	----	------------------------------------	------------	------------------	----

- sql code:

UPDATE Journal

SET ImpactFactor = 50.25

WHERE ISSN = 'J001';

4) Update an author's affiliation:

- sql code:

UPDATE Author

SET Affiliation = 'University of California, Berkeley'

WHERE AuthorID = 2;

5) Count total authors:

- sql code:

SELECT COUNT(*) AS TotalAuthors FROM Author;

```
234 • SELECT J.JournalName, P.PublisherName
235 FROM Journal J
236 JOIN Publisher P ON J.PublisherID = P.PublisherID;
237
```

JournalName	PublisherName
Nature Genetics	Nature Publishing Group
Cell	Springer
Immunity	Elsevier
Neuron	Elsevier
Development	Cambridge University Press
Molecular Cell	Elsevier
Genomics Journal	Wiley
Biochemistry Reports	American Chemical Society
Microbiology Today	Royal Society of Chemistry
Pharmacology Review	Taylor & Francis

```
225 • SELECT COUNT(*) AS TotalAuthors FROM Author;
226
227
```

TotalAuthors
10

6) Average citations per research paper:

- sql code:

SELECT AVG(Citations) AS AvgCitations FROM Research;

7) Show research titles with their authors:

- sql code:

ISSN	JournalName	ImpactFactor	Quarter	PublisherID	Country	StartDate	VolumePublicationRate	OpenAccessStatus
J001	Nature Genetics	50.25	Q1	5	UK	1990-01-01	Monthly	1

```
SELECT R.Title, A.NameAuthor
```

```
FROM Research R
```

```
JOIN ResearchAuthor RA ON R.ResearchID =  
RA.ResearchID
```

```
JOIN Author A ON RA.AuthorID = A.AuthorID;
```

	JournalName	ImpactFactor
▶	Nature Genetics	50.25
	Cell	38.45
	Immunity	25.12
	Neuron	20.34

8) Show journals with their publishers:

- sql code:

```
SELECT J.JournalName, P.PublisherName
```

```
FROM Journal J
```

```
JOIN Publisher P ON J.PublisherID = P.PublisherID;
```

```
243 • SELECT * FROM TopJournals;  
244
```

Result Grid	Filter Rows:
JournalName	ImpactFactor
▶ Nature Genetics	50.25
Cell	38.45
Immunity	25.12
Neuron	20.34

9) Create a view for top journals:

- sql code:

```
CREATE VIEW TopJournals AS
```

```
SELECT JournalName, ImpactFactor
```

```
FROM Journal
```

```
WHERE ImpactFactor > 20;
```

10) Query the view:

- sql code:

```
SELECT * FROM TopJournals;
```

```
227 • SELECT AVG(Citations) AS AvgCitations FROM Research;  
228
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
AvgCitations			
▶ 103.5000			

```
229 • SELECT R.Title, A.NameAuthor  
230 FROM Research R  
231 JOIN ResearchAuthor RA ON R.ResearchID = RA.ResearchID  
232 JOIN Author A ON RA.AuthorID = A.AuthorID;  
233
```

Result Grid	Filter Rows:	Export:	Wrap Cell Content:
Title	NameAuthor		
▶ Gene Editing Advances	Alice Johnson		
Gene Editing Advances	Bob Smith		
Immune Response Mechanisms	Carol White		
Immune Response Mechanisms	David Brown		
Neural Pathways	Eva Green		
Neural Pathways	Frank Black		
Drug Development	Grace Lee		
Drug Development	Henry Adams		
Cell Division	Ivy Chen		
Cell Division	Jack Wilson		

ResearchID	Title	Abstract	PublicationDate	Citations	BiomedicalFieldID	JournalID	PublisherID	PublicationType
1	Gene Editing Advances	Study on CRISPR applications.	2020-05-01	120	2	J001	1	Journal
2	Immune Response Mechanisms	Exploring T-cell activation.	2019-03-15	85	3	J003	2	Journal
3	Neural Pathways	Mapping brain circuits.	2021-07-20	60	4	J004	3	Book
4	Drug Development	New pharmacological targets.	2018-11-10	95	5	J010	4	Journal
5	Cell Division	Mechanisms of mitosis.	2017-01-05	150	1	J002	5	Journal
6	Microbial Resistance	Antibiotic resistance study.	2022-02-12	70	6	J009	6	Book
7	Protein Folding	Biochemical analysis.	2016-09-30	110	7	J008	7	Book
8	Molecular Pathways	Signal transduction research.	2020-12-01	80	8	J006	8	Journal
9	Developmental Stages	Embryonic growth study.	2015-04-25	65	9	J005	9	Journal
10	Genomic Sequencing	Advances in sequencing tech.	2023-06-18	200	10	J007	10	Journal
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

ResearchID	AuthorID	PublisherID	PublisherName	PublisherID	PaymentMethod
1	1	1	Springer	1	Cash
1	2	2	Elsevier	2	Bank Transfer
2	3	3	Wiley	3	Online Payment
2	4	4	Taylor & Francis	4	Cash
3	5	5	Nature Publishing Group	5	Online Payment
3	6	6	Oxford University Press	6	Bank Transfer
4	7	7	Cambridge University Press	7	Cash
4	8	8	American Chemical Society	8	Online Payment
5	9	9	IEEE	9	Bank Transfer
5	10	10	Royal Society of Chemistry	10	Cash
NULL	NULL	NULL	NULL	NULL	NULL

5. Test data: at least 10 rows per table:

6. Explanation: rationale for design choices & implementation notes:

ISSN	JournalName	ImpactFactor	Quarter	PublisherID	Country	StartDate	VolumePublicationRate	OpenAccessStatus	ISSN	FieldID
J001	Nature Genetics	45.67	Q1	5	UK	1990-01-01	Monthly	1	J002	1
J002	Cell	38.45	Q1	1	USA	1974-01-01	Monthly	1	J001	2
J003	Immunity	25.12	Q1	2	USA	1994-01-01	Monthly	0	J003	3
J004	Neuron	20.34	Q2	2	USA	1988-01-01	Monthly	0	J004	4
J005	Development	15.89	Q2	7	UK	1953-01-01	Monthly	1	J010	5
J006	Molecular Cell	18.45	Q2	2	USA	1997-01-01	Monthly	0	J009	6
J007	Genomics Journal	12.34	Q3	3	USA	1985-01-01	Quarterly	1	J008	7
J008	Biochemistry Reports	10.56	Q3	8	USA	1960-01-01	Quarterly	0	J006	8
J009	Microbiology Today	8.90	Q4	10	UK	1970-01-01	Annually	1	J005	9
J010	Pharmacology Review	9.45	Q4	4	USA	1980-01-01	Quarterly	0	J007	10
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

AuthorID	NameAuthor	Qualification	Affiliation	JobTitle	Email	HIndex	AuthorID	FieldID	FieldID	FieldName
1	Alice Johnson	PhD	Harvard University	Professor	alice@harvard.edu	45	5	1	1	Cell Biology
2	Bob Smith	MD	Stanford University	Researcher	bob@stanford.edu	32	1	2	2	Genetics
3	Carol White	PhD	Oxford University	Lecturer	carol@oxford.edu	28	2	3	3	Immunology
4	David Brown	PhD	MIT	Postdoc	david@mit.edu	15	3	4	4	Neuroscience
5	Eva Green	PhD	Cambridge University	Professor	eva@cambridge.edu	50	4	5	5	Pharmacology
6	Frank Black	MD	Yale University	Researcher	frank@yale.edu	20	6	6	6	Microbiology
7	Grace Lee	PhD	UCLA	Lecturer	grace@ucla.edu	18	7	7	7	Biochemistry
8	Henry Adams	PhD	Johns Hopkins	Professor	henry@jh.edu	40	8	8	8	Molecular Biology
9	Ivy Chen	PhD	University of Toronto	Researcher	ivy@utoronto.edu	22	9	9	9	Developmental Biology
10	Jack Wilson	PhD	University of Chicago	Postdoc	jack@uchicago.edu	12	10	10	10	Genomics
NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

- Each table represents a single entity (Research, Author, Journal, Publisher, BiomedicalField).
- Redundant data is avoided by using ****junction tables**** for many-to-many relationships (e.g., ResearchAuthor, AuthorField).
- This ensures consistency and scalability.

-Primary Keys:

- Every table has a Primary Key (e.g., ResearchID, AuthorID, ISSN, PublisherID).
- Guarantees uniqueness of records and efficient indexing.

-Foreign Keys:

- Used to enforce relationships:
- Research → Journal (JournalID).
- Research → BiomedicalField (BiomedicalFieldID).
- Journal → Publisher (PublisherID).
- Ensures referential integrity: you cannot insert a research paper linked to a non-existent journal.

-Composite Keys:

- Junction tables (ResearchAuthor, AuthorField, JournalField, PublisherPublicationType, PublisherPaymentMethod) use ****composite primary keys****.
- Prevents duplicate associations (e.g., the same author being linked twice to the same research).

-Data Types:

- 'VARCHAR' for text fields (names, emails, journal names).
- 'INT' for numeric identifiers and counts.
- 'DECIMAL' for impact factors (precise scientific values).
- 'DATE' for publication dates.
- 'BOOLEAN' for open access status.

-Flexibility:

- Biomedical fields are stored in a separate table, allowing easy expansion.
- Publishers can have multiple publication types and payment methods via junction tables.
- Authors and journals can belong to multiple fields, reflecting real-world complexity.

-Database Creation:

- Creates a dedicated database for biomedical research.
- Ensures all tables and queries are scoped to this database.

- Table Creation:

- Each 'CREATE TABLE' defines structure, constraints, and relationships.
- SQL executes these sequentially, building the schema layer by layer.

- Foreign keys require referenced tables to exist first (e.g., Publisher must exist before Journal).

- Data Insertion:

- ``INSERT INTO`` populates tables with sample records.
- SQL validates constraints:
- Primary keys must be unique.
- Foreign keys must reference valid IDs.
- Unique constraints (like Author.Email) prevent duplicates.

- Data Manipulation:

- ``UPDATE`` modifies existing records.
- ``DELETE`` removes records (with caution — cascading deletes may be needed in production).
- ``SELECT`` retrieves data, with filters (``WHERE``), aggregates (``AVG``, ``COUNT``), and joins.

- Views:

- ``CREATE VIEW`` stores reusable queries as virtual tables.
- Useful for analytics (e.g., HighlyCitedResearch, TopJournals).

- Joins:

- Joins connect related tables:
- Research ↔ Author (via ResearchAuthor).
- Research ↔ Journal.
- Journal ↔ Publisher.
- SQL executes joins by matching foreign keys to primary keys.

Questions:

1. You must model authors ↔ research (many-to-many). What cross-reference tables and keys would you create?

Composite Primary Key: to connect them, you use the Research_Authors table. It strips the many-to-many complexity into two separate One-to-Many (1:M) relationships.

What it does is combine both IDs into a single, unique identifier for each row. And why it matters is that it prevents data duplication. An author cannot be credited twice for the same research paper.

Foreign Keys:

Research_ID Foreign Key: Links the row to a valid paper in the Research table.

Author_ID Foreign Key: Links the row to a valid scientist in the Author table.

ON DELETE CASCADE: If a paper or an author profile is deleted from the master tables, their corresponding connection rows in this bridge table are automatically wiped out, preventing "orphaned" reference data.

Payload Attributes:

It tracks the order of authorship (e.g., 1 for First Author/Lead Researcher, 2 for Co-Author, etc.). And why it matters is that it's a specific piece of information that does not belong to the Author alone, nor does it belong to the Research paper alone—it only exists when the two intersect.

2. Describe a query to list all authors in the “Immunology” field with their total citation counts for 2023.

SELECT

Author_ID, Name AS Author_Name,

COUNT(Citing_Research_ID) AS Total_Citations_2023

FROM

Author a

JOIN

Author_Interests ai ON a.Author_ID = ai.Author_ID

JOIN

Research_Authors ra ON a.Author_ID = ra.Author_ID

JOIN

Citations c ON ra.Research_ID = c.Cited_Research_ID

JOIN

Research r_citing ON c.Citing_Research_ID = r_citing.Research_ID

WHERE

ai.Field_Name = 'Immunology'

AND YEAR(r_citing.Pub_Date) = 2023

GROUP BY

a.Author_ID, a.Name

ORDER BY

Total_Citations_2023 DESC;

Identifying the Target Authors (JOIN Author_Interests)

The query starts at the Author table and immediately joins Author_Interests. The WHERE ai.Field_Name = 'Immunology' clause acts as a gatekeeper, instantly filtering out any researchers who do not specialize in Immunology.

Linking Authors to their Written Work (JOIN Research_Authors)

We bridge the filtered authors to the papers they authored using the Research_Authors bridge table. This establishes a pool of Research_IDs belonging to our immunologists.

Finding the Citations (JOIN Citations)

This is the trickiest part of the query. We join the Citations table by matching the author's paper (ra.Research_ID) to the **Cited_Research_ID** column. This tells us every time one of our immunologist's papers was referenced by another paper.

Isolating the Year 2023 (JOIN Research r_citing)

A citation doesn't inherently hold a timestamp—the *citing paper* does. Therefore, we must perform a self-style join back to the Research table, aliasing it as r_citing.

- We match `c.Citing_Research_ID = r_citing.Research_ID`.
- The `YEAR(r_citing.Pub_Date) = 2023` filter ensures we **only count citations that occurred in papers published in the year 2023**.

Aggregating and Counting (GROUP BY & COUNT)

Finally, `COUNT(c.Citing_Research_ID)` counts the total number of incoming citations. Because we use `GROUP BY a.Author_ID, a.Name`, the database collapses the rows so that each author appears exactly once alongside their cumulative 2023 citation tally. Any author with zero citations in 2023 under this strict inner join schema would be omitted (if you wanted to display them with a 0, you would swap the citation joins to `LEFT JOINs`).

3. When a new citation is recorded, how would you update the `Research.citations` count—via trigger or application logic?

```
CREATE TRIGGER After_New_Citation
AFTER INSERT ON Citations
FOR EACH ROW
BEGIN
    UPDATE Research
    SET Citation_Count = Citation_Count + 1
    WHERE Research_ID = NEW.Cited_Research_ID;
END;
```

Guaranteed Data Integrity: No matter how data enters your database—whether through your web application, a python data-scraping script, or a developer running manual SQL queries in the command line—the citation count will always be perfectly accurate.

Centralized Logic: You don't have to rewrite the increment logic in multiple places if you build different microservices or applications that interface with this database.